

BANCO DE DADOS I

MATERIAL DE AULA

Fácil assim



PROF. ENILDO

Sumário

CAPÍTULO 1 — CONCEITOS FUNDAMENTAIS	2
Exercícios	6
CAPÍTULO 2 — INTRODUÇÃO A BANCOS DE DADOS	7
Exercícios	10
Projeto da Disciplina Fase 1— Sistema de Vendas	10
CAPÍTULO 3 — SISTEMA GERENCIADOR DE BANCO DE DADOS (SGBD)	11
Exercícios	13
Projeto da Disciplina Fase 2 - Sistema de Vendas.....	14
CAPÍTULO 4 — ARQUITETURA CLIENTE/SERVIDOR.....	14
Projeto da Disciplina Fase 3 - Sistema de Vendas.....	16
CAPÍTULO 5 — MODELAGEM CONCEITUAL	17
Exercícios 5	19
Projeto da Disciplina Fase 4 - Sistema de Vendas.....	19
CAPÍTULO 6 — RELACIONAMENTOS E CARDINALIDADE.....	20
Exercícios	23
Projeto da Disciplina Fase 5 - Sistema de Vendas.....	23
CAPÍTULO 7 — FERRAMENTAS CASE E MODELAGEM NO brModelo	23
Exercícios	25
Projeto da Disciplina — Sistema de Vendas	25
CAPÍTULO 8 — Especialização e generalização.	26
CAPÍTULO 9 — DOMÍNIO	32
CAPÍTULO 10 — TIPOS DE DADOS E NULL	34
Linguagem SQL Padrão ANSI	36
Sublinguagens SQL	36

BANCO DE DADOS I

Modelagem e SQL com MySQL

Ferramentas utilizadas: MySQL, MySQL Workbench e brModelo

CAPÍTULO 1 — CONCEITOS FUNDAMENTAIS

1.1 Dados x Informação

Antes de estudar banco de dados, é importante entender a diferença entre **dado** e **informação**.

Dado

É um valor bruto, sem interpretação.

Exemplos:

- 3500
- João
- 10/03/2026

Esses valores isolados não possuem significado completo.

Informação

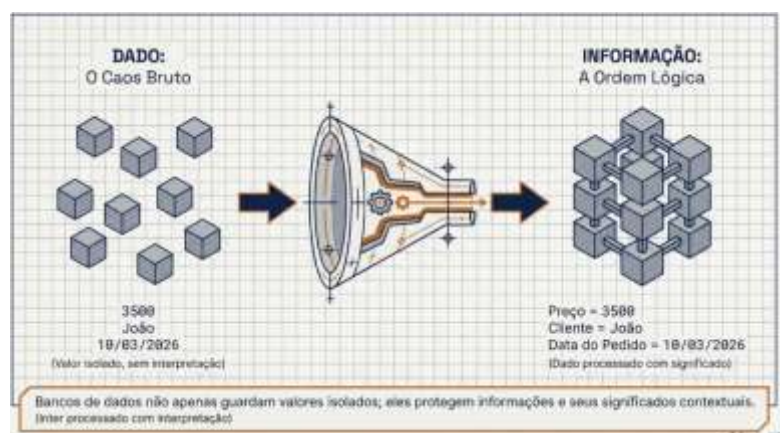
É o dado processado com significado.

Exemplos:

- Preço do produto = 3500
- Nome do cliente = João
- Data do pedido = 10/03/2026

Portanto:

- Dado → valor bruto
- Informação → dado interpretado



1.2 Sistemas que armazenam dados

Nem todos os sistemas utilizam banco de dados. Alguns armazenam dados permanentemente e outros não.

Sistemas que utilizam banco de dados

- Sistema bancário
- Sistema escolar
- Sistema de vendas
- Redes sociais
- Comércio eletrônico



Esses sistemas precisam armazenar informações como:

- clientes
- produtos
- pedidos
- pagamentos

Sistemas que normalmente **NÃO** utilizam banco de dados

- Calculadora simples
- Relógio digital
- Conversor de unidades simples
- Bloco de notas offline básico

Esses sistemas executam tarefas sem necessidade de armazenamento estruturado.

1.3 Aplicação Cliente

A aplicação cliente é o sistema utilizado pelo usuário para acessar o banco de dados.

Exemplos:

- Sistema de vendas
- Aplicativo mobile
- Sistema web
- MySQL Workbench

Fluxo de funcionamento:

Usuário → Aplicação → Banco de Dados

O usuário não acessa o banco diretamente.
Ele utiliza uma aplicação que envia comandos ao banco.

1.4 Quem acessa o Banco de Dados

Existem diferentes tipos de usuários que interagem com o banco.

Usuário final

Utiliza o sistema no dia a dia.

Exemplo:

- vendedor
- atendente
- operador de caixa

Não acessa diretamente o banco.



Desenvolvedor

Cria o sistema que acessa o banco de dados.

Responsabilidades:

- desenvolver a aplicação
- criar consultas SQL
- implementar regras de negócio

DBA — Administrador de Banco de Dados

Responsável pela administração do banco.

Funções:

- criar banco de dados
- controlar acesso
- realizar backup
- monitorar desempenho
- garantir segurança
- realizar manutenção

1.5 Servidor físico x Servidor lógico

Servidor físico

É o equipamento onde o banco de dados está armazenado.

Exemplos:

- computador
- servidor dedicado
- máquina virtual

É o hardware.

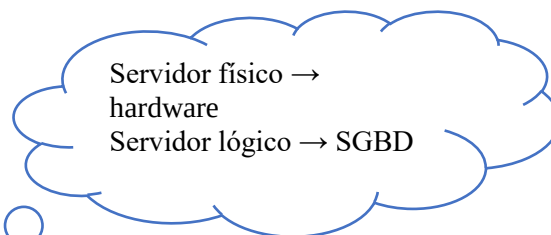
Servidor lógico

É o software que gerencia o banco de dados.

Exemplos:

- MySQL
- SQL Server
- Oracle
- PostgreSQL

É o SGBD.



1.6 Data Center

Um data center é um local onde vários servidores ficam armazenados.

Características:

- climatização
- controle de energia
- segurança física
- internet de alta velocidade
- redundância de equipamentos

Empresas que possuem grandes data centers:

- Google
- Amazon
- Microsoft

1.7 Computação em Nuvem

Na computação em nuvem, o banco de dados não fica na empresa, mas em servidores remotos.

Exemplos:

- AWS (Amazon Web Services)
- Microsoft Azure
- Google Cloud

Vantagens:

- não precisa comprar servidor
- alta disponibilidade
- escalabilidade
- backup automático



1.8 Estrutura geral de um sistema de banco de dados

Um sistema de banco de dados normalmente funciona assim:

Usuário → Aplicação → SGBD → Banco de Dados → Servidor

Esse modelo é chamado de arquitetura cliente/servidor, que será estudado nos próximos capítulos.



Exercícios

Questionário

1. Explique com suas palavras a diferença entre dado e informação.
2. Cite dois exemplos de sistemas que utilizam banco de dados.
3. Cite um exemplo de sistema que não utiliza banco de dados.
4. Qual é a função do DBA?
5. Qual a diferença entre servidor físico e servidor lógico?
6. O que é computação em nuvem?
7. Quem acessa diretamente o banco de dados:
 - a) Usuário final
 - b) Aplicação
 - c) Teclado
 - d) Monitor

Projeto da Disciplina — Sistema de Vendas

Imagine que você precisa desenvolver um sistema para uma loja.

Responda:

1. Quais dados precisarão ser armazenados?
2. Quem utilizará o sistema?
3. O sistema será local ou em nuvem?
4. Quem será o responsável pela administração do banco?

Essas respostas serão utilizadas no desenvolvimento do projeto ao longo da disciplina.

CAPÍTULO 2 — INTRODUÇÃO A BANCOS DE DADOS

2.1 O que é um Banco de Dados

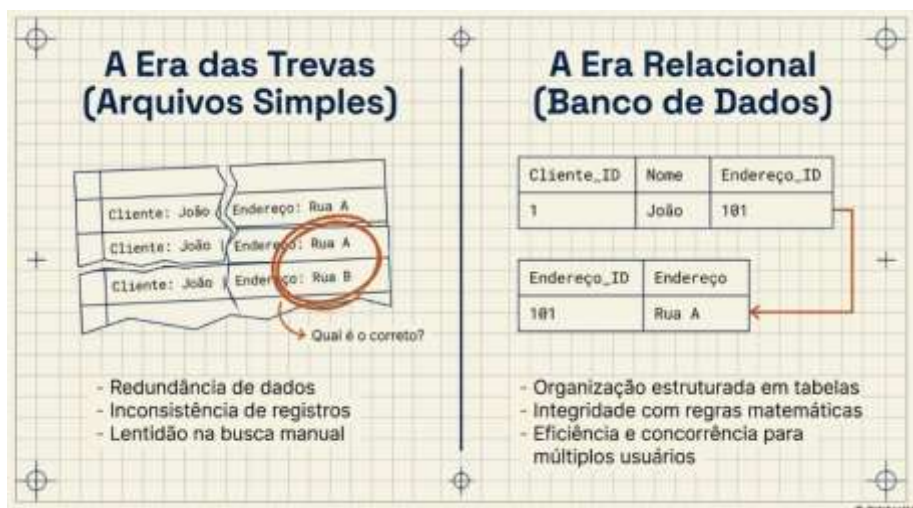
Um banco de dados é uma coleção organizada de dados armazenados eletronicamente em um sistema computacional.

Esses dados representam informações utilizadas por empresas e sistemas, como:

- clientes
- produtos
- pedidos
- funcionários
- pagamentos
- estoque

Um banco de dados permite:

- armazenar dados
- consultar dados
- atualizar dados
- excluir dados
- proteger dados



2.2 Exemplo do mundo real

Imagine uma loja que registra suas vendas.

Cliente	Produto	Quantidade	Valor
João	Notebook	1	3500
Ana	Mouse	2	50

Essas informações precisam ser armazenadas e organizadas.

Se não houver banco de dados, surgem problemas como:

- dificuldade para localizar dados
- duplicação de informações
- erros de registro
- dificuldade para gerar relatórios

2.3 Armazenamento em arquivos

Antes dos bancos de dados, os sistemas utilizavam arquivos simples.

Exemplo:

Cliente	Endereço
João	Rua A
João	Rua A
João	Rua A

Isso gera redundância de dados.

2.4 Problemas do armazenamento em arquivos

Redundância de dados

O mesmo dado aparece várias vezes.

Inconsistência

Informações diferentes para o mesmo registro.

Exemplo:

Cliente	Endereço
João	Rua A
João	Rua B

Qual é o correto?

Dificuldade de acesso

Buscar dados manualmente é lento.

Exemplos:

- localizar vendas de um cliente
- calcular total de vendas
- listar produtos mais vendidos

2.5 Vantagens do Banco de Dados

- Organização
Os dados são estruturados em tabelas.
- Integridade
Regras garantem consistência.
- Segurança
Controle de acesso aos dados.
- Eficiência
Consultas rápidas.
- Compartilhamento
Vários usuários acessam simultaneamente.

2.6 Exemplos de uso de Banco de Dados

Bancos

Armazenam:

- contas
- saldos
- transações

Comércio eletrônico

Armazenam:

- produtos
- pedidos
- clientes

Redes sociais

Armazenam:

- usuários
- mensagens
- fotos



2.7 Componentes de um Sistema de Banco de Dados

Dados

Informações armazenadas.

Exemplo:

- nomes
- preços
- datas

Hardware

Equipamentos físicos.

Exemplo:

- servidores
- discos

Software

Sistema gerenciador de banco de dados (SGBD)

Exemplos:

- MySQL
- SQL Server
- Oracle

Usuários

Pessoas que utilizam o sistema.

- DBA
- desenvolvedor
- usuário final

2.8 Exemplo de estrutura de banco de dados

Tabela Cliente

id_cliente	nome
1	João
2	Maria

Tabela Produto

id_produto	nome	preço
1	Notebook	3500
2	Mouse	50

Exercícios

- a) Explique o que é um banco de dados.
- b) Cite três exemplos de sistemas que utilizam banco de dados.
- c) Explique o problema da redundância de dados.
- d) Quais são as vantagens do uso de banco de dados?
- e) Quais são os componentes de um sistema de banco de dados?

Projeto da Disciplina Fase 1— Sistema de Vendas

Com base no sistema de vendas, responda:

- Quais tabelas serão necessárias?
- Quais dados cada tabela terá?
- Por que usar banco de dados nesse sistema?

CAPÍTULO 3 — SISTEMA GERENCIADOR DE BANCO DE DADOS (SGBD)

3.1 O que é um SGBD

Um Sistema Gerenciador de Banco de Dados (SGBD) é um software responsável por criar, organizar, armazenar e manipular dados em um banco de dados.



Em outras palavras, o SGBD é o programa que permite que usuários e sistemas interajam com os dados.

Sem um SGBD seria muito difícil:

- controlar grandes volumes de dados
- garantir segurança
- manter consistência
- permitir acesso simultâneo

3.2 Funções de um SGBD

Armazenamento de dados

Organiza os dados em tabelas.

Manipulação de dados

Permite:

- inserir dados
- atualizar dados
- excluir dados
- consultar dados

Essas operações são feitas com SQL.

Controle de acesso

Define quem pode:

- ler dados
- inserir dados
- modificar dados

Exemplo:

administrador
usuário comum



Integridade dos dados

Garante consistência.

- Exemplo:
Não permitir pedido para cliente inexistente.

Controle de concorrência

Permite vários usuários acessando ao mesmo tempo.

Recuperação de falhas

Recupera dados após:

- queda de energia
- falha do sistema



3.3 Exemplos de SGBDs

Alguns dos mais utilizados:

- MySQL
- PostgreSQL
- SQL Server
- Oracle
- MongoDB

3.4 SGBD x SGBDR

SGBD

Sistema que gerencia qualquer banco de dados.

SGBDR

Sistema que gerencia banco de dados relacional.



Exemplo:

Sistema	Tipo
MySQL	SGBDR
PostgreSQL	SGBDR
SQL Server	SGBDR
MongoDB	SGBD (NoSQL)

Resumo:

Todo SGBDR é um SGBD

Nem todo SGBD é relacional

3.5 O Modelo Relacional

O modelo relacional organiza dados em tabelas.

Cada tabela possui:

- colunas (atributos)
- linhas (registros)

Exemplo:

Tabela Cliente

id_cliente	nome	telefone
1	João	9999-1111
2	Maria	9888-2222

Cada linha representa um registro.



3.6 Estrutura de uma tabela

Uma tabela é composta por:

- Colunas (atributos)
- Representam características.

Exemplos:

- nome
- telefone
- email

Linhas (registros)

- Representam dados completos.

Chave Primária

- É o campo que identifica cada registro de forma única.

Exemplo:

id_cliente	nome
1	João
2	Maria

O campo id_cliente é a chave primária.

3.7 SGBD utilizado na disciplina

Nesta disciplina utilizaremos:

MySQL

Características:

- gratuito
- código aberto
- amplamente utilizado
- rápido
- utilizado em sistemas web

3.8 Ferramenta de gerenciamento

Para utilizar o MySQL usaremos:

MySQL Workbench

Permite:

- criar bancos de dados
- criar tabelas
- executar SQL
- visualizar dados
- administrar servidor

3.9 Primeiro comando SQL

Abra o MySQL Workbench e execute:

```
SELECT 'Banco de Dados I';
```

Esse comando retorna um texto confirmando o funcionamento.

Exercícios

- O que é um SGBD?
- Cite dois exemplos de SGBDs.
- Qual a diferença entre SGBD e SGBDR?
- O que é chave primária?
- Qual o SGBD utilizado nesta disciplina?



Exercício Prático

Abra o MySQL Workbench e execute:

`SELECT NOW();`

Observe o resultado exibido.

Projeto da Disciplina Fase 2 - Sistema de Vendas

- Com base no sistema:
- Qual SGBD será utilizado?
- Quantas tabelas serão necessárias?
- Cada tabela terá uma chave primária?

CAPÍTULO 4 — ARQUITETURA CLIENTE/SERVIDOR

4.1 O que é Arquitetura Cliente/Servidor

A arquitetura cliente/servidor é um modelo em que as tarefas são divididas entre dois componentes:

Cliente Servidor

Nesse modelo:

- o cliente solicita
- o servidor processa
- o servidor responde

Esse tipo de arquitetura é muito utilizado em:

- sistemas web
- sistemas bancários
- aplicativos corporativos
- sistemas de vendas

4.2 Funcionamento da Arquitetura

O funcionamento ocorre em três etapas:

- O cliente envia uma solicitação
- O servidor processa
- O servidor retorna a resposta

No contexto de banco de dados:

- o cliente envia um comando SQL
- o servidor executa
- o servidor retorna o resultado

4.3 Cliente no Sistema de Banco de Dados

O cliente é a aplicação utilizada pelo usuário.

Exemplos:

- MySQL Workbench
- sistema web
- aplicativo mobile
- sistema desktop

O cliente permite:

- executar SQL
- visualizar dados
- inserir dados
- consultar informações

4.4 Servidor no Sistema de Banco de Dados

O servidor é responsável por:

- armazenar dados
- executar comandos SQL
- gerenciar conexões
- garantir segurança

Exemplo:

MySQL Server

- O servidor processa as solicitações enviadas pelos clientes.

4.5 Exemplo de Funcionamento

Usuário executa o comando:

`SELECT * FROM cliente;`

Fluxo:

1. MySQL Workbench envia o comando
2. MySQL Server executa
3. O servidor retorna o resultado

Resultado:

id_cliente	nome
1	João
2	Maria

4.6 Vantagens da Arquitetura Cliente/Servidor

Centralização dos dados

Todos os dados ficam no servidor.

- Segurança
Controle de acesso.
- Escalabilidade
Vários clientes podem acessar.
- Facilidade de manutenção
Atualizações feitas no servidor.

4.7 Tipos de Arquitetura Cliente/Servidor

Dois níveis (Two-Tier)

Cliente se comunica diretamente com o banco.

Aplicação → Banco de Dados

Utilizado em sistemas simples.

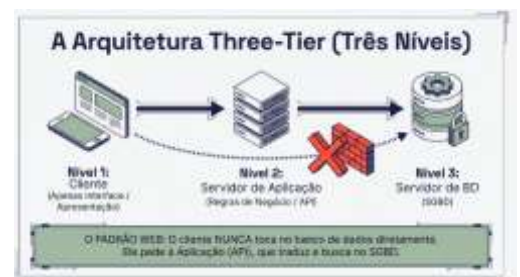


Três níveis (Three-Tier)

Existe uma camada intermediária.

Cliente → Aplicação → Banco de Dados

Utilizado em sistemas web.



4.8 Exemplo de Arquitetura em Três Camadas

Cliente: navegador

Aplicação: servidor web

Banco: MySQL

Fluxo:

Usuário → Navegador → Servidor Web → Banco de Dados

4.9 Conexão com o Banco de Dados

Para acessar o banco é necessário:

- host
- porta
- usuário
- senha

Exemplo:

Host: localhost

Porta: 3306

Usuário: root

Exercícios

- a) Explique o que é arquitetura cliente/servidor.
- b) Qual é o papel do cliente?
- c) Qual é a função do servidor?
- d) Cite um exemplo de cliente de banco de dados.
- e) Qual a diferença entre two-tier e three-tier?

Exercício Prático

Execute no MySQL Workbench:

```
SELECT NOW();
```

Esse comando confirma a comunicação entre cliente e servidor.

Projeto da Disciplina Fase 3 - Sistema de Vendas

Responda:

- O sistema será cliente/servidor?
- Qual será o cliente?
- Qual será o servidor?
- O sistema será two-tier ou three-tier?

CAPÍTULO 5 — MODELAGEM CONCEITUAL

5.1 O que é Modelagem de Dados

A modelagem de dados é o processo de organizar e estruturar as informações que serão armazenadas em um banco de dados.

Antes de criar tabelas, é necessário entender:

- quais dados serão armazenados
- como os dados se relacionam
- quais regras devem ser aplicadas

A modelagem conceitual representa o sistema de forma visual, facilitando o entendimento.

5.2 Modelo Conceitual

O modelo conceitual representa os dados do ponto de vista do negócio.

Ele não se preocupa com:

- SQL
- tipos de dados
- implementação

Ele descreve apenas:

- entidades
- atributos
- relacionamentos

5.3 Entidades

Entidades são objetos do mundo real que serão armazenados no banco de dados.

Exemplos:

- Cliente
- Produto
- Pedido
- Funcionário



Uma entidade normalmente se transforma em uma tabela.

Exemplo:

Entidade: Cliente

Tabela: cliente



cliente
codigo: int
nome: varchar(50)

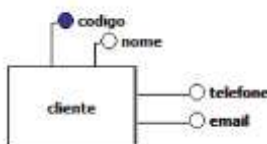
5.4 Atributos

Atributos são as características de uma entidade.

Exemplo:

Entidade Cliente

- id_cliente
- nome
- telefone
- email



Esses atributos se tornam colunas da tabela.

Tipo de Atributo	Característica Principal	Exemplo	Observação
Simple	Indivisível	CPF	Valor único
Composto	Divisível em partes	Endereço	Rua, nº, cidade
Multivalorado	Possui vários valores	Telefone	Vira tabela no relacional
Derivado	Calculado a partir de outro	Idade	Evita redundância
Identificador (Chave)	Identifica unicamente	ID, CPF	Fica sublinhado
Opcional	Pode não ter valor (NULL)	Complemento endereço	Não obrigatório

5.5 Relacionamentos

Relacionamentos representam associações entre entidades.

Exemplo:

Cliente realiza Pedido

Nesse caso:

Cliente está relacionado com Pedido



Outro exemplo:

Pedido contém Produto

Relacionamentos mostram como os dados se conectam.



5.6 Exemplo completo

Entidade Cliente

id_cliente

nome

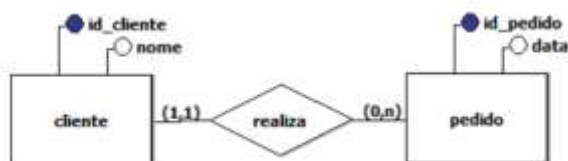
Entidade Pedido

id_pedido

data

Relacionamento:

Cliente realiza Pedido



5.7 Diagrama Entidade-Relacionamento (DER)

A modelagem conceitual é representada por um DER.

Componentes:

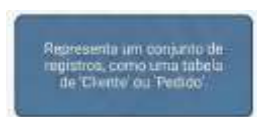
Entidade → retângulo

Atributo → oval

Relacionamento → losango ou linha

Exemplo conceitual:

Cliente — realiza — Pedido



5.8 Exemplo do Sistema de Vendas

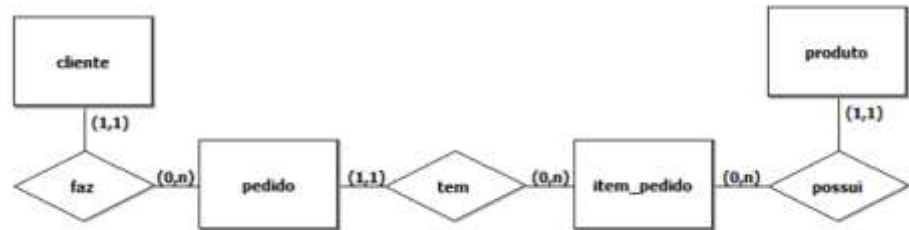
Entidades principais:

- Cliente
- Produto
- Pedido
- Item_Pedido

Relacionamentos:

Cliente faz Pedido

Pedido possui Produto



5.9 Vantagens da Modelagem Conceitual

- Facilita entendimento do sistema
- Evita erros no projeto
- Reduz redundância
- Organiza dados
- Permite planejamento antes da implementação

Exercícios 5

Questionário

1. O que é modelagem de dados?
2. O que é uma entidade?
3. O que é um atributo?
4. O que é um relacionamento?
5. Cite duas entidades de um sistema escolar.

Projeto da Disciplina Fase 4 - Sistema de Vendas

Identifique:

Entidades do sistema:

- Cliente
- Produto
- Pedido
- Item_Pedido

Agora responda:

- Quais atributos cada entidade terá?
- Qual entidade se relaciona com Pedido?
- Pedido pode conter mais de um produto?

CAPÍTULO 6 — RELACIONAMENTOS E CARDINALIDADE

O que são Relacionamentos

Relacionamentos representam a associação entre duas ou mais entidades.

Eles mostram como os dados estão conectados.

Exemplo:

Cliente realiza Pedido

Nesse caso:

- Cliente está relacionado com Pedido



Tipos de Relacionamentos

Existem três tipos principais:

- Um para Um (1:1)
- Um para Muitos (1:N) “Ambos os lados”
- Muitos para Muitos (N:N)

Relacionamento Um para Um (1:1)

Uma ocorrência da entidade A se relaciona com apenas uma ocorrência da entidade B.

Entidades:

- Pessoa
- Passaporte

Relacionamento:

Uma pessoa possui um passaporte, e cada passaporte pertence a apenas uma pessoa.



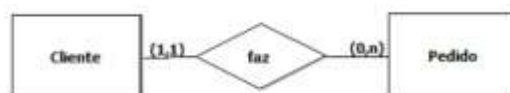
Relacionamento Um para Muitos (1:N)

Uma ocorrência da entidade A pode se relacionar com várias ocorrências da entidade B.

Exemplo:

Cliente realiza Pedido

- Um cliente pode fazer vários pedidos
- Um pedido pertence a apenas um cliente



Relacionamento Muitos para Muitos (N:N)

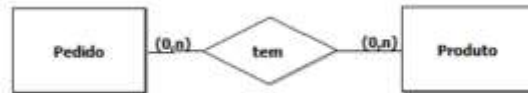
Várias ocorrências da entidade A se relacionam com várias ocorrências da entidade B.

Exemplo:

Pedido contém Produto

- Um pedido pode ter vários produtos
- Um produto pode aparecer em vários pedidos

Esse tipo não é implementado diretamente.



É necessário criar uma tabela intermediária.

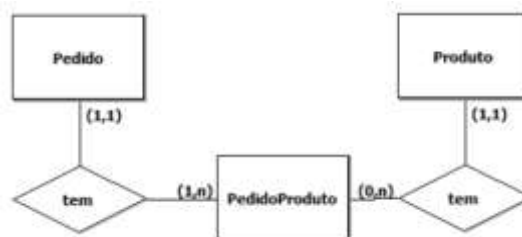
Entidade Associativa

Para resolver o relacionamento N:N criamos uma nova entidade.

Exemplo: (Pedido N:N Produto)

Criamos: (Item_Pedido, Tabela Item_Pedido, **id_pedido id_produto Quantidade**)

Isso transforma em: Pedido 1:N Item_Pedido , Produto 1:N Item_Pedido



Cardinalidade

A cardinalidade indica quantas ocorrências participam do relacionamento.

Exemplos:

1:1 → um para um

1:N → um para muitos

N:N → muitos para muitos

Também pode indicar obrigatoriedade.

Exemplo:

Cliente 0..N Pedido

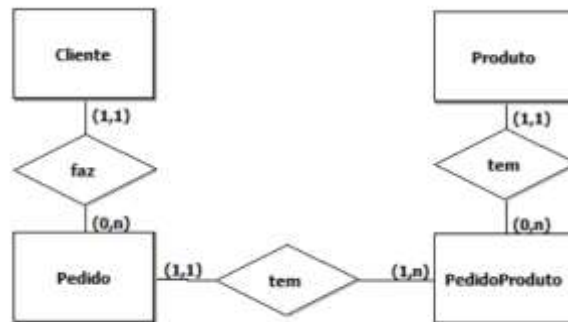
Um cliente pode não ter pedido.

Exemplo Sistema de Vendas

Cliente 1:N Pedido

Pedido N:N Produto

Produto 1:N Item_Pedido

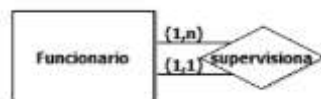


Auto relacionamento

Ocorre quando uma entidade se relaciona com ela mesma.

Exemplo:

Funcionário supervisiona Funcionário



Grau do relacionamento

Relacionamento **binário**

Entre duas entidades.

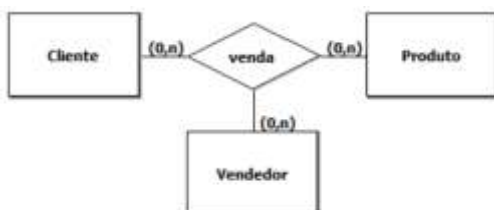
Cliente — Pedido



Relacionamento ternário

Entre três entidades.

Cliente — Produto — Vendedor



Exercícios

- a) O que é relacionamento?
- b) Explique relacionamento 1:N.
- c) Explique relacionamento N:N.
- d) O que é entidade associativa?
- e) O que é cardinalidade?

Projeto da Disciplina Fase 5 - Sistema de Vendas

Defina:

1. Cliente e Pedido → qual relacionamento?
2. Pedido e Produto → qual relacionamento?
3. Será necessária tabela intermediária?
4. Qual o nome da tabela?

CAPÍTULO 7 — FERRAMENTAS CASE E MODELAGEM NO brModelo

O que são Ferramentas CASE

Ferramentas CASE (Computer-Aided Software Engineering) são softwares que auxiliam no desenvolvimento de sistemas.

No contexto de banco de dados, elas permitem:

- criar diagramas
- modelar entidades
- definir relacionamentos
- gerar código SQL
- documentar o banco

Essas ferramentas ajudam a organizar o projeto antes da implementação.

Vantagens das Ferramentas CASE

- facilitam a modelagem
- reduzem erros
- organizam o projeto
- geram documentação
- permitem visualização gráfica

Exemplos de Ferramentas CASE

Algumas ferramentas utilizadas:

- MySQL Workbench
- brModelo
- DBDesigner
- ERDPlus
- Lucidchart

Nesta disciplina utilizaremos:

brModelo

O que é o brModelo

O brModelo é uma ferramenta utilizada para criar diagramas entidade-relacionamento.

Características:

- gratuita
- voltada para ensino
- fácil de usar
- permite modelagem conceitual e lógica

Pode ser utilizada:

- versão desktop
- versão web

Componentes no brModelo

Entidade

Representa tabelas.

Atributo

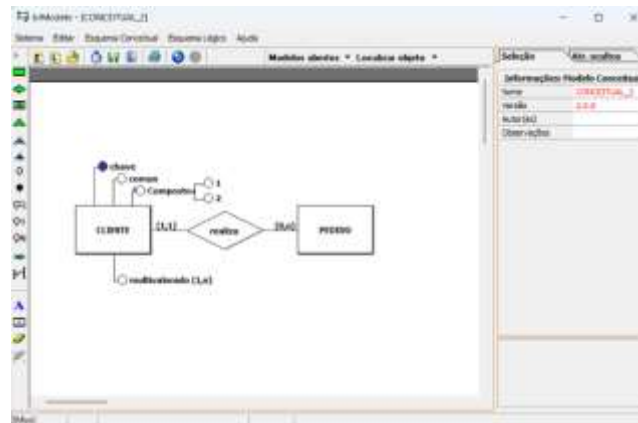
Representa colunas.

Relacionamento

Representa ligação entre entidades.

Cardinalidade

Representa quantidade de relacionamentos.



Exemplo de Modelagem

Sistema de Vendas

Entidades:

- Cliente
- Produto
- Pedido



Relacionamentos:

Cliente realiza Pedido

Pedido contém Produto

Etapas de Modelagem com brModelo

1. Identificar entidades
2. Criar atributos
3. Definir chave primária
4. Criar relacionamentos
5. Definir cardinalidade
6. Gerar modelo lógico

Benefícios da Modelagem

- ✓ organização dos dados
- ✓ evita redundância
- ✓ facilita criação das tabelas
- ✓ melhora entendimento do sistema

Exercícios

- a) O que é uma ferramenta CASE?
- b) Cite um exemplo de ferramenta CASE.
- c) Qual ferramenta será utilizada na disciplina?
- d) Para que serve o brModelo?
- e) Quais elementos existem no brModelo?

Projeto da Disciplina — Sistema de Vendas

Modele no brModelo:

Entidades:

- Cliente
- Produto
- Pedido
- Item_Pedido

Defina:

- atributos
- relacionamentos
- cardinalidade

CAPÍTULO 8 — Especialização e generalização.

Especialização e generalização são técnicas de modelagem de dados que permitem organizar e estruturar os dados de forma hierárquica, representando entidades com diferentes níveis de granularidade.

O que é especialização?

A especialização é o processo de dividir uma entidade genérica em entidades mais específicas, cada uma com suas próprias características e propriedades. As entidades especializadas herdam as características da entidade genérica, mas também possuem características próprias que as diferenciam.

Exemplo

- **Entidade genérica:** (Veículo)
- **Entidades especializadas:** (Carro, Motocicleta, Caminhão...)

Cada entidade especializada possui características próprias que a diferenciam das demais. Por exemplo, um carro possui portas e assentos, enquanto uma motocicleta possui guidão e baú.

O que é generalização?

A generalização é o processo de agrupar entidades com características semelhantes em uma única entidade genérica. A entidade genérica representa as características comuns às entidades especializadas.

Exemplo

- **Entidade genérica:** (Animal)
- **Entidades especializadas:** (Cachorro, Gato, Cavalo, ...)

Todas as entidades especializadas são animais, portanto, herdam as características comuns aos animais, como ter pelos, respirar e se alimentar.

Tipos de especialização

Existem dois tipos de especialização:

- **Especialização total:** Cada ocorrência da entidade genérica deve pertencer a uma única entidade especializada.
- **Especialização parcial:** Uma ocorrência da entidade genérica pode pertencer a nenhuma, uma ou mais entidades especializadas.

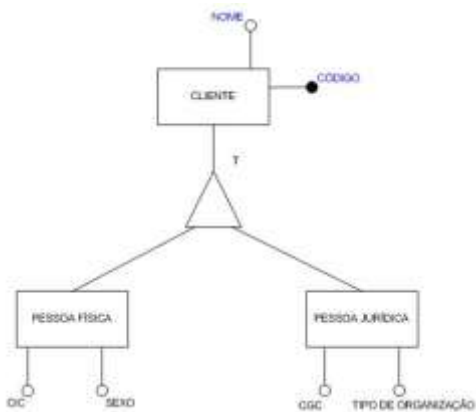
Exemplos

Imagine uma empresa de seguros que vende seguros para seus clientes que podem ser tanto cidadãos como empresas.

Neste caso teríamos a situação abaixo:

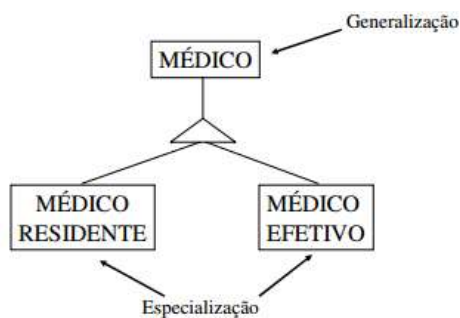
CLIENTE
PESSOA JURÍDICA
PESSOA FÍSICA

No Diagrama Entidade e Relacionamentos este conceito pode ser representado assim:



Em uma empresa de Planos de Saúde poderíamos ter a seguinte situação:

- PACIENTE
- MÉDICO
- MÉDICO RESIDENTE
- MÉDICO EFETIVO



Cada uma dessas categorias, além de características comuns, possui atributos distintos.

Generalização/Especialização

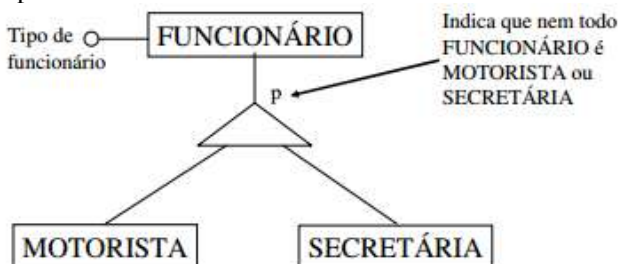
Através deste conceito é possível atribuir propriedades particulares a um subconjunto das ocorrências (especializadas) de uma entidade genérica ou entidade Pai.

Herança de propriedades: cada ocorrência da entidade especializada possui, além de seus próprios atributos e relacionamentos, todos os atributos da entidade generalizada.

Tipos de Generalização/Especialização

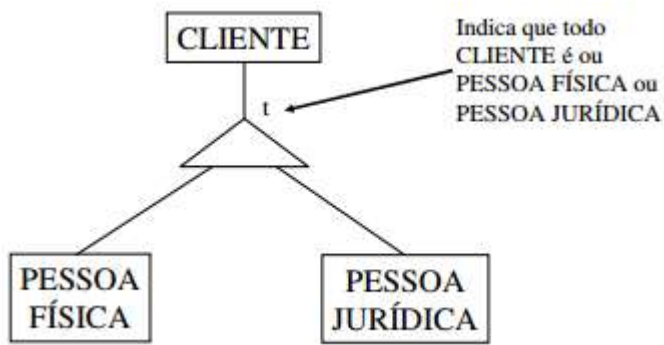
A Generalização/Especialização pode ser classificada em dois tipos:

Parcial: nem toda ocorrência da entidade genérica possui uma ocorrência correspondente em uma entidade especializada.



No exemplo acima, nem todo funcionário é motorista e nem todo funcionário é secretária. Pode haver funcionários que não sejam nem motorista e nem secretária.

Total: para toda ocorrência da entidade genérica existe sempre uma ocorrência em uma das entidades especializadas.



No exemplo acima, todo cliente ou é uma pessoa física ou uma pessoa jurídica. Não existe a possibilidade de haver um cliente que não seja pessoa física OU pessoa jurídica.

Vantagens da especialização e generalização

- **Melhor organização dos dados:** As entidades são organizadas de forma hierárquica, facilitando a compreensão e o gerenciamento dos dados.
- **Redução da redundância:** As características comuns às entidades especializadas são armazenadas apenas na entidade genérica, evitando redundância no banco de dados.
- **Maior flexibilidade:** O modelo de dados pode ser facilmente adaptado para novas necessidades.

Desvantagens da especialização e generalização

- **Aumento da complexidade do modelo:** O modelo de dados pode se tornar mais complexo, dificultando a compreensão e a manutenção.
- **Dificuldade de implementação:** A implementação da especialização e generalização pode ser complexa em alguns sistemas de gerenciamento de banco de dados.

Quando usar especialização e generalização?

A especialização e generalização devem ser utilizadas quando:

- Existem entidades com características comuns que podem ser agrupadas em uma única entidade genérica.
- Existem entidades que podem ser divididas em entidades mais específicas.

É importante avaliar as vantagens e desvantagens da especialização e generalização antes de utilizá-las em um modelo de dados.

CAPÍTULO 9 — NORMALIZAÇÃO

O que é Normalização

A normalização é o processo de organizar os dados em tabelas para evitar:

- redundância
- inconsistência
- anomalias de atualização

Ela melhora:

- organização
- integridade
- eficiência

Redundância de Dados

Redundância ocorre quando o mesmo dado aparece várias vezes.

Exemplo:

Cliente	Produto	Endereço
João	Notebook	Rua A
João	Mouse	Rua A

O endereço está repetido.

Problemas:

- ocupa espaço
- risco de inconsistência

Objetivo da Normalização

- eliminar duplicação
- organizar dados
- separar entidades
- melhorar consistência

Primeira Forma Normal (1FN)

Regra:

Cada campo deve possuir apenas um valor.

Exemplo incorreto:

Cliente	Telefones
João	9999,8888

Exemplo correto:

Cliente	Telefone
João	9999
João	8888

Segunda Forma Normal (2FN)

Regra:

Todos os atributos devem depender da chave primária completa.

Exemplo incorreto:

id_pedido id_produto nome_produto

nome_produto depende apenas de id_produto.

Solução:

Separar em outra tabela.

Terceira Forma Normal (3FN)

Regra:

Não deve haver dependência transitiva.

Exemplo incorreto:

| id_cliente | cidade | estado |
estado depende de cidade.

Solução:

Criar tabela Cidade.

Benefícios da Normalização

- reduz redundância
- evita inconsistência
- melhora organização
- facilita manutenção

Desnormalização

Desnormalização é o processo inverso.

Usada para:

- melhorar performance
- reduzir joins

Desvantagens:

- aumenta redundância
- dificulta manutenção

Dicionário de dados em banco de dados

Um dicionário de dados é um repositório central de metadados que descreve os objetos de um banco de dados, como tabelas, colunas, chaves e índices.

Ele fornece informações sobre:

- **Nome do objeto:** Nome da tabela, coluna, chave ou índice.
- **Descrição:** Uma breve descrição do que o objeto representa.
- **Tipo de dado:** O tipo de dado armazenado na coluna.
- **Tamanho:** O tamanho do campo em bytes.
- **Restrições:** Restrições de integridade, como NOT NULL, UNIQUE e FOREIGN KEY.
- **Relacionamentos:** Relacionamentos entre as tabelas.
- **Proprietário:** O usuário responsável pelo objeto.
- **Data de criação:** A data em que o objeto foi criado.
- **Data de modificação:** A data em que o objeto foi modificado pela última vez.

Benefícios de um dicionário de dados

- Melhora a documentação do banco de dados: Facilita a compreensão da estrutura e do funcionamento do banco de dados.
- Facilita o gerenciamento do banco de dados: Permite que os administradores e desenvolvedores gerenciem os objetos de forma mais eficiente.
- Melhora a qualidade dos dados: Ajuda a garantir a consistência e a integridade dos dados.
- Reduz a redundância de dados: Evita que as mesmas informações sejam armazenadas em vários lugares.
- Melhora a comunicação entre os usuários do banco de dados: Facilita a comunicação entre os usuários do banco de dados, pois fornece um vocabulário comum para descrever os objetos.

Ferramentas para criar um dicionário de dados

Ferramentas de modelagem de dados: A maioria das ferramentas de modelagem de dados possui recursos para criar dicionários de dados.

SGBDs: Alguns SGBDs, como o Oracle e o SQL Server, possuem ferramentas integradas para criar dicionários de dados.

Ferramentas de terceiros: Existem diversas ferramentas de terceiros para criar dicionários de dados.

Recomendações

Mantenha o dicionário de dados atualizado.

Defina um padrão para a documentação dos objetos.

Utilize ferramentas para facilitar a criação e a manutenção do dicionário de dados.

Treine os usuários do banco de dados sobre como usar o dicionário de dados.

Junto com o modelo de entidade e relacionamento, é necessário que se mantenha um documento com a explicação de todos os objetos nele criados. Este documento, que pode ser chamado de dicionário de dados, permite que os analistas obtenham informações sobre todos os objetos do modelo de forma textual, contendo explicações por vezes difíceis de incluir no diagrama. É válido lembrar que o objetivo do documento é ser claro e consistente.

Exemplo de dicionário de dados:

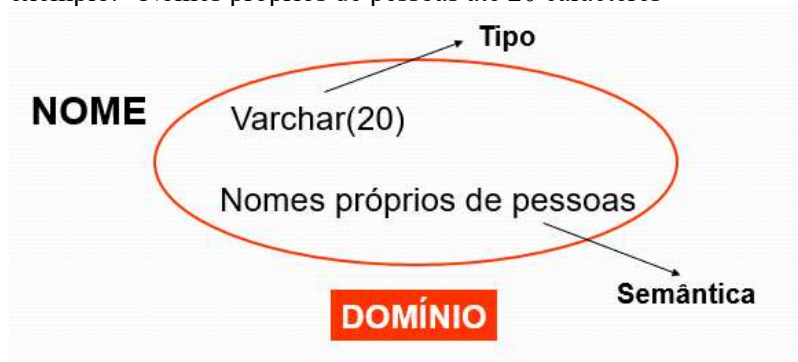
Entidade: Cliente				
Atributo	Classe	Domínio	Tamanho	Descrição
Codigo_cliente	Determinante	Numérico		
Nome	Simples	Texto	50	
Telefone	Multivalorado	Texto	50	Valores sem as máscaras de entrada
Cidade	Simples	Texto	50	
data_nascimento	Simples	Data		Formato dd/mm/aaaa

CAPÍTULO 10— DOMÍNIO

- Conjunto de valores permitidos para um dado
- Possui uma descrição física e outra semântica.
- A descrição física identifica o tipo e o formato dos valores que compõem o domínio a descrição semântica ajuda na interpretação de seus valores

exemplo: “Números de telefone válidos no Brasil (99)9999-9999”

exemplo: “Nomes próprios de pessoas até 20 caracteres”



Conceito de tabelas

Uma tabela pode ser composta de uma lista completa de indivíduos em uma lista de discussão, incluindo seus endereços e números de telefone. As tabelas são frequentemente classificadas por finalidade ou aplicação a que se destinam. Alguns exemplos comuns incluem listas de discussão, tabelas de controle de qualidade, tabelas de estoque, ou tabelas de documentos. Tabelas também podem ser classificadas pelo grau de permanência que elas têm. Tabelas de transação são apenas temporárias, enquanto tabelas mestres são utilizadas por mais tempo.

Uma tabela é a estrutura dentro do seu banco de dados que contém os dados organizados em campos (colunas) e registros (linhas).

Construção de projeto lógico de banco de dados

Todo projeto de um sistema de aplicação para banco de dados necessita de um coração, um centro nervoso. A modelagem de um sistema pela abordagem Entidade-Relacionamento representa esse ponto central no Projeto Conceitual de um sistema.

A utilização de uma abordagem correta de metodologia orientada a banco de dados envolve a estruturação nos três níveis de visão de dados, ou seja, três etapas na execução de um projeto: conceitual, lógico e físico.



Um banco de dados bem-estruturado:

- Economiza espaço em disco ao eliminar dados redundantes.
- Mantém a exatidão e a integridade dos dados.
- Oferece acesso aos dados de maneiras úteis.

Criar um banco de dados eficiente e útil é uma questão de seguir o processo adequado, incluindo as fases a seguir:

- Análise de requisitos, ou identificação do objetivo do banco de dados
- Organizando dados em tabelas
- Especificando chaves primárias e analisando relações
- Normalizando para padronizar as tabelas

CAPÍTULO 11 — TIPOS DE DADOS E NULL

Tipos de Dados SQL Padrão ANSI

O SQL Padrão ANSI define um conjunto de tipos de dados básicos para garantir a interoperabilidade entre diferentes sistemas de gerenciamento de banco de dados (SGBDs).

Estes tipos de dados podem ser categorizados em:

1. Tipos Numéricos:

Inteiros:

- SMALLINT: Números inteiros com 2 bytes (de -32768 a 32767).
- INTEGER: Números inteiros com 4 bytes (de -2147483648 a 2147483647).
- BIGINT: Números inteiros com 8 bytes (de -9223372036854775808 a 9223372036854775807).

Reais:

- FLOAT: Números de ponto flutuante com precisão simples (7 dígitos).
- DOUBLE: Números de ponto flutuante com precisão dupla (15 dígitos).
- DECIMAL: Números de ponto flutuante com precisão e escala especificadas.

2. Tipos de Caractere:

- CHAR(n): String fixa de caracteres com tamanho n.
- VARCHAR(n): String variável de caracteres com tamanho máximo n.
- TEXT: String longa de caracteres (sem limite de tamanho).
- NCHAR(n): String fixa de caracteres Unicode com tamanho n.
- NVARCHAR(n): String variável de caracteres Unicode com tamanho máximo n.

3. Tipos de Data e Hora:

- DATE: Data no formato AAAA-MM-DD.
- TIME: Hora no formato HH:MM:SS.
- DATETIME: Data e hora no formato AAAA-MM-DD HH:MM:SS.
- TIMESTAMP: Data e hora com precisão de frações de segundo.

4. Tipos Especiais:

- BIT: Valor booleano (TRUE/FALSE).
- BINARY(n): Array de bytes com tamanho n.
- VARBINARY(n): Array variável de bytes com tamanho máximo n.

5. Tipos de Dados Nulos:

- NULL: Indica um valor ausente ou desconhecido.

Observações

A maioria dos SGBDs suportam tipos de dados adicionais além dos tipos padrão ANSI.

A sintaxe específica para declarar e usar tipos de dados pode variar entre diferentes SGBDs.

Consulte a documentação do seu SGBD para obter mais detalhes sobre os tipos de dados disponíveis.

Modelo lógico

O Modelo Lógico tem por objetivo representar as estruturas que irão armazenar os dados dentro de um Banco de Dados, a partir deste momento é que são definidas com maior propriedade as entidades e os seus atributos.

• Regras de derivação

Regras de derivação descrevem atributos cujo valor depende de outro atributo ou de um cálculo. No diagrama, o símbolo RD é utilizado para identificar esse tipo de regra. A descrição completa da fórmula utilizada para compor o valor do atributo deve ser documentada no dicionário de dados.

Por exemplo, para modelar o valor total de uma compra, vamos acrescentar dois atributos no relacionamento COMPRA: quantidade comprada e total (o atributo preço já está modelado na entidade Livro). Esse último contém uma regra de derivação: seu valor é calculado através da soma da quantidade de livros comprados multiplicado pelo preço de venda e subtraído pelo desconto de promoção, caso o cliente tenha direito.

• Regras de Restrição

O modelo relacional, ao definir conceitos como Tabela, Coluna, Nulo, Domínio, Chave Primária e Chave Estrangeira, deixa implícitas algumas regras fundamentais para a manutenção da consistência da base de dados. Elas são chamadas de Regras de Integridade e tratam dos cuidados que analistas, projetistas e programadores devem observar ao implementar as rotinas de Inclusão, Alteração e Exclusão de dados nos objetos.

Desnormalização

A desnormalização é uma técnica usada em bancos de dados relacionais para aumentar o desempenho de consultas, principalmente consultas que envolvem várias tabelas relacionadas. Ela funciona através da introdução de redundância de dados.

- **Performance de consultas:** É a rapidez com que um banco de dados consegue responder a uma consulta. Consultas que envolvem várias tabelas (através de junções) geralmente são mais demoradas do que consultas em tabelas únicas.
- **Redundância de dados:** É quando o mesmo dado é armazenado em mais de um local no banco de dados. Normalmente, bancos de dados relacionais são projetados para minimizar a redundância para evitar problemas de consistência de dados. Porém, em alguns casos, a introdução de uma certa redundância estratégica pode melhorar significativamente a performance.

Aqui estão alguns cenários onde a desnormalização pode ser útil:

- **Consultas frequentes que envolvem junções:** Se você tem uma consulta que é executada com muita frequência e envolve a junção de várias tabelas, desnormalizar os dados necessários para essa consulta em uma tabela única pode melhorar drasticamente o desempenho.
- **Tabelas de dimensão em data warehouses:** Data warehouses são usados para armazenar dados históricos para análise. Tabelas de dimensão em data warehouses costumam ser desnormalizadas para melhorar o desempenho de consultas analíticas complexas.

É importante ressaltar que a desnormalização deve ser feita com cautela. Alguns pontos negativos a se considerar são:

- **Aumento do tamanho do banco de dados:** Devido à redundância, o banco de dados vai ocupar mais espaço de armazenamento.
- **Manutenção mais complexa:** Como o mesmo dado pode estar armazenado em vários lugares, a atualização desse dado precisa ser feita em todos os locais para manter a consistência, o que pode aumentar a complexidade da manutenção do banco de dados.

Resumindo, a desnormalização é uma técnica que pode ser usada para otimizar o desempenho de consultas em bancos de dados relacionais, mas deve ser usada com cuidado para evitar problemas com o tamanho e a manutenção do banco de dados.

CAPÍTULO 12 - Linguagem SQL Padrão ANSI

Existem inúmeras linguagens no mercado de TI, linguagens de programação orientadas a objeto, estruturadas, de marcação de texto entre outras. A linguagem voltada para banco de dados é a SQL (Linguagem de consulta estruturada, em português) é uma linguagem que todo programador, técnico ou administrador de banco de dados deve conhecer. Sua aplicação é extremamente ampla no mercado de banco de dados e programação.

A linguagem SQL (Structure query Language - Linguagem de Consulta Estruturada) é a linguagem padrão ANSI (American National Standards Institute - Instituto Nacional de Padronização Americano) para a operação em bancos de dados relacionais. A linguagem SQL foi criada para atender a todos os bancos de dados relacionais e permitir que usuários possam acessar qualquer banco usando a mesma base de conhecimento.

O padrão ANSI define os seguintes elementos da linguagem SQL:

- **Comandos básicos:** SELECT, INSERT, UPDATE, DELETE etc.
- **Funções:** Funções matemáticas, de agregação, de manipulação de strings etc.
- **Tipos de dados:** Inteiros, strings, datas etc.
- **Operadores:** Operadores aritméticos, lógicos, comparativos etc.
- **Restrições:** NOT NULL, UNIQUE, PRIMARY KEY etc.

Sublinguagens SQL

A linguagem SQL é composta por diversas sublinguagens que permitem realizar tarefas específicas:

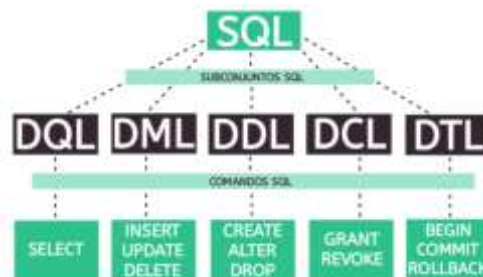
Data Definition Language (DDL): Define a estrutura do banco de dados, como tabelas, colunas e chaves.

Data Manipulation Language (DML): Permite manipular dados, como inserir, atualizar e excluir.

Data Query Language (DQL): Permite consultar dados, como selecionar e ordenar.

Data Control Language (DCL): Controla o acesso aos dados, como permissões e usuários.

Transaction Control Language (TCL): Controla transações, como commit e rollback.



O SQL Padrão ANSI define um conjunto básico de comandos e funcionalidades que todos os SGBDs devem suportar. As sublinguagens SQL estendem o padrão ANSI com funcionalidades específicas de cada SGBD.

Aprender o SQL Padrão ANSI é importante para:

- Escrever código SQL mais portátil: O código escrito em SQL Padrão ANSI pode ser executado em diferentes SGBDs sem necessidade de modificações significativas.
- Facilitar o aprendizado de sublinguagens: As sublinguagens SQL são extensões do padrão ANSI, portanto, aprender o padrão ANSI facilita o aprendizado das sublinguagens.
- Melhorar a compreensão da linguagem SQL: O padrão ANSI define a base da linguagem SQL, portanto, compreendê-lo ajuda a entender melhor a linguagem como um todo.

O SQL Padrão ANSI é a base da linguagem SQL e é importante para escrever código SQL mais portátil, facilitar o aprendizado de sublinguagens e melhorar a compreensão da linguagem como um todo.